

Creación de una red neuronal feedforward para un problema de clasificación

Esequiel Armeria and Gonzalo Ramos

Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Ciudad Universitaria, 5000 Córdoba, Argentina

(Dated: 14 de noviembre de 2025)

I. ABSTRACT

En este trabajo se llevo a cabo la implementación de una red neuronal feedforward multicapa (MLP) para la clasificación de imágenes del conjunto *Fashion-MNIST*. Se implementaron sus funciones de entrenamiento y de prueba, así como una batería de experimentos que nos llevaran a mejorar los hiperparametros. El objetivo de este trabajo fue reconocer diez categorías de prendas de vestir mediante un modelo entrenado con descenso por gradiente (optimizador Adam) en PyTorch. La arquitectura constó de dos capas ocultas con 512 y 256 neuronas, activaciones ReLU y regularización *dropout*. El modelo alcanzó una precisión promedio del **89,11 %** en el conjunto de test, mostrando un buen compromiso entre simplicidad y desempeño.

II. INTRODUCCIÓN

Las redes neuronales artificiales feedforward son modelos computacionales inspirados en el funcionamiento del cerebro humano [2]. Están compuestas por capas de neuronas artificiales interconectadas, donde cada neurona de una capa se conecta con todas las neuronas de la capa anterior y a su vez con todas las de la capa siguiente, a este tipo de conexión se la denomina *fully connected*. Estas conexiones hacen que la información fluya solo hacia un lado, hacia adelante, posibilitando el aprendizaje de relaciones complejas entre las variables de entrada y salida a partir de ejemplos, lo que las convierte en herramientas potentes para la clasificación de datos.

Este aprendizaje se puede realizar de múltiples formas, una supervisada, no supervisada, por refuerzos y semi-supervisada [8]. ¿Pero cual es la diferencia entre las dos?, esta radica en la información que pueden ver el modelo durante su entrenamiento. Mientras en los aprendizajes supervisados, los datos ya tiene una etiqueta preestablecida, en los casos no supervisados esperamos que el algoritmo encuentre por nosotros una relación entre estos. Por ejemplo, separarlos en grupos con una determinada característica en común (Clustering). El caso estudiado en este informe forma parte del primer tipo de aprendizaje, donde el algoritmo encuentra relaciones entre estos datos y las etiquetas dadas. A este paso se lo suele denominar como "entrenamiento" del modelo.

Mientras entrenamos nuestro modelo, podemos ir midiendo su desempeño. Para esto se definen métricas que

dependen de la tarea que este va a desempeñar. En caso de una clasificación, una de las más utilizadas es la de exactitud (*accuracy*), que es la proporción de ejemplos para los cuales el modelo predijo la salida correcta según las etiquetas con las que lo entrenamos. En el caso que estemos realizando una regresión, una posible métrica es el RMSE (*Root Mean Squared Error*) que es la medición de la raíz cuadrada de la diferencia promedio entre los valores predichos y los valores reales en nuestros datos. Esto nos va a decir que tan bien entrenado esta nuestro modelo. Pero no todo se limita a que nuestras métricas de entrenamiento sean los mejor posibles, El objetivo principal del ML es la generalización, osea que nuestro modelo pueda predecir correctamente nuevos ejemplos sin un label asignado previamente. Para ver esto, realizamos un testeo posterior a que el entrenamiento termine. Esta fase, se basa en mostrarle ejemplos que no vio durante el entrenamiento y ver con cuanta exactitud los predijo. Las métricas suelen ser iguales a las que usamos para el entrenamiento.

En este trabajo se realizo una red neuronal artificial para crear un clasificador de imágenes de prendas de ropa utilizando el dataset Fashion-MNIST ([3]). Se hizo experimentos con sus hiperparametros para observar cuanto y como cambia el comportamiento del modelo, tanto en su entrenamiento como en su testeo.

III. TEORÍA

Una red neuronal feedforward multicapa (MLP, por sus siglas en inglés) es un modelo computacional inspirado en la estructura del cerebro humano. Su función principal es aprender relaciones no lineales entre las variables de entrada y las de salida mediante un proceso de entrenamiento particular.

A. Estructura

Una red neuronal feedforward multicapa (MLP) está organizada en capas de neuronas artificiales que se conectan de manera densa, es decir, cada neurona de una capa está conectada con todas las neuronas de la capa siguiente. Estas conexiones están asociadas a pesos sinápticos que cuantifican la importancia de cada entrada sobre la activación de una neurona.

1. Capa Entrada

La capa de entrada constituye el punto de acceso de los datos a la red neuronal. Cada neurona de esta capa representa una característica (*feature*) del conjunto de datos y se conecta con todas las neuronas de la primera capa oculta mediante pesos sinápticos w_{ij} , transfiriendo así la información hacia las capas internas de la red.

A esta capa pueden ingresarle dos conjuntos de datos distintos, según el propósito del proceso: el conjunto de *entrenamiento*, utilizado cuando se entrena la red neuronal, y el conjunto de *testeo*, empleado una vez entrenado el modelo para evaluar su desempeño con datos no vistos previamente y analizar su capacidad de generalización.

En este trabajo se utiliza el conjunto de datos *Fashion-MNIST* [3], compuesto por imágenes de prendas de vestir en escala de grises de 28×28 píxeles. Cada imagen se aplanan en un vector de 784 componentes, de modo que cada neurona de la capa de entrada recibe como valor la intensidad normalizada de un píxel.

La función principal de esta capa es actuar como un canal de transmisión: recibe los valores de las variables de entrada y los propaga hacia las capas subsiguientes. A diferencia de las capas ocultas, la capa de entrada no realiza operaciones matemáticas ni aplica funciones de activación; simplemente presenta los datos a la red para su procesamiento posterior.

2. Capa Oculta

Las capas ocultas son el núcleo del aprendizaje en una red neuronal. Cada neurona en estas capas transforma la información recibida aplicando una función de activación no lineal sobre la combinación lineal de las entradas. Cada neurona en estas capas calcula una combinación lineal de las salidas de la capa anterior mediante los pesos sinápticos w_i y un término de sesgo (bias) b . El bias es un valor constante que se añade al resultado de la suma ponderada de las entradas de una neurona, antes de aplicar la función de activación. Dota a la red neuronal de una mayor flexibilidad para modelar relaciones complejas y capturar patrones que solo con los pesos no se podrían representar, lo que ayuda a evitar el subajuste (underfitting) y mejora la precisión general del modelo. Matemáticamente, lo que sucede en cada neurona es:

$$z_j = \sum_{i=1}^n w_{ji}x_i + b_j$$

donde:

- x_i es la salida de la neurona de la capa i anterior
- w_{ij} es el peso de la conexión entre la neurona de la capa anterior i y la neurona j de la capa siguiente

- b es el bias de la neurona j

En este trabajo, se utilizó la función ReLU (Rectified Linear Unit) [4], definida como:

$$f(z) = \max(0, z)$$

Esta función tiene un comportamiento particular:

- Si $z > 0$, la salida es z (la neurona se activa completamente).
- Si $z < 0$, la salida es 0 (la neurona se inhibe o "permanence dormida").

Este mecanismo introduce no linealidad al modelo, lo que le permite aprender relaciones complejas entre las variables de entrada. A diferencia de funciones como la sigmoide o la tangente hiperbólica, ReLU no satura los gradientes en valores grandes, evitando que las actualizaciones de los pesos se vuelvan insignificantes durante el entrenamiento (problema conocido como vanishing gradient).

Durante el proceso de aprendizaje, los pesos se ajustan de modo que las neuronas ReLU tiendan a activar selectivamente ciertas regiones del espacio de características. Esto genera representaciones jerárquicas de los datos: en las primeras capas, las neuronas aprenden patrones simples (bordes o texturas), mientras que en capas más profundas reconocen estructuras más abstractas (formas de prendas, patrones de sombras, etc.). Además de este proceso, también se aplican variables de regularización para evitar el sobre ajuste. En nuestro caso utilizaremos (*dropout*), que se basa en "pagar" neuronas aleatoriamente.

3. Capa Salida

La capa de salida es la última capa de la red neuronal. Recibe como entrada los valores (activaciones) que provienen de la última capa oculta, los combina mediante sus pesos y bias, y produce la predicción final del modelo. El número de neuronas en una capa de salida, va a depender del problema que estemos intentando resolver.

- Si es un problema de regresión, típicamente la salida será de una sola neurona que produce un número continuo.
- Si es un problema de clasificación, en una clasificación binaria será una sola neurona con una activación sigmoide, y si una multi clase, serán las cantidad de neuronas igual a la cantidad de labels.

Una vez calculadas las combinaciones lineales de la salida de la última capa oculta para cada neurona, estas pasan por una función de activación distinta. En nuestro caso, es Softmax [5]:

$$a_j = \frac{e^{z_k}}{\sum_{k=1}^K e^{z_j}}$$

donde K es el número total de clases y j la neurona. Esto convierte los valores z_j en probabilidades normalizadas entre 0 y 1 que suman 1.

Durante el entrenamiento, se calcula el error entre las salidas predichas a_j y las verdaderas (target) y_j , usando la función de pérdida, en nuestro caso Cross-Entropy [7]

$$L = - \sum_{j=1}^K y_j \ln(a_j)$$

Luego, en el proceso de *backpropagation* [6], se calcula el gradiente del error respecto de cada peso:

$$\frac{\partial L}{\partial w_{ji}} = \delta_j a_i$$

donde:

$$\delta_j = (a_j - y_j)$$

representa el *error local* de la neurona j en la capa de salida. Los pesos se actualizan mediante el método de *descenso del gradiente* según:

$$w_{ji}^{(t+1)} = w_{ji}^{(t)} - \eta \frac{\partial L}{\partial w_{ji}}$$

donde η es la *tasa de aprendizaje* (*learning rate*), que controla la magnitud de la actualización de los pesos en cada iteración.

IV. RESULTADOS

Como mencionamos previamente, el objetivo de este informe es encontrar un modelo predictivo utilizando redes neuronales para la clasificación de imágenes de diferentes prendas de ropa, haciendo uso del dataset Fashion-MNIST. Partiremos de una arquitectura base con hiperparámetros pre-establecidos que iremos modificando para conocer como reacciona la red a estos. Nuestros parámetros base son:

- Cantidad de capas intermedias: 2
- Cantidad de neuronas por capa: Primera capa 128, segunda capa: 64
- Learning rate : 0.001
- Optimizador: SGD
- Función de pérdida: Cross Entropy
- Tamaño de lote: 100
- Dropout: 0.2

A. Experimento 1

En este experimento, llevamos a cabo la variación de nuestro learning rate entre rangos altos y muy bajos, así como también se utiliza el optimizador Adam

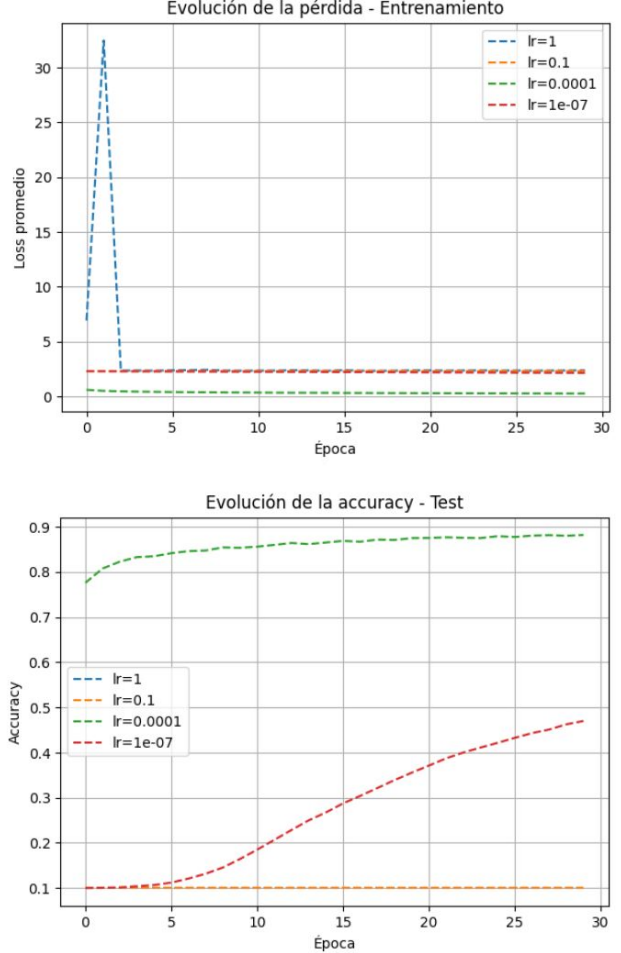


Figura 1. Evolución la pérdida y exactitud durante el entrenamiento y prueba con distintas **Tasas de aprendizaje**. Vemos que en los extremos, presentan el comportamiento esperado. Cuando la tasa de aprendizaje es muy alta encontramos picos y cuando es muy pequeña el aprendizaje es muy lento, los errores casi no varían. Cuando verificamos como se comporta el modelo entrenado con estas tasas con datos no vistos, vemos que generaliza mal, como era esperado.

B. Experimento 2

En este experimento, veremos distinta cantidad de tamaños de lote y su relación con distintas cantidad de épocas.

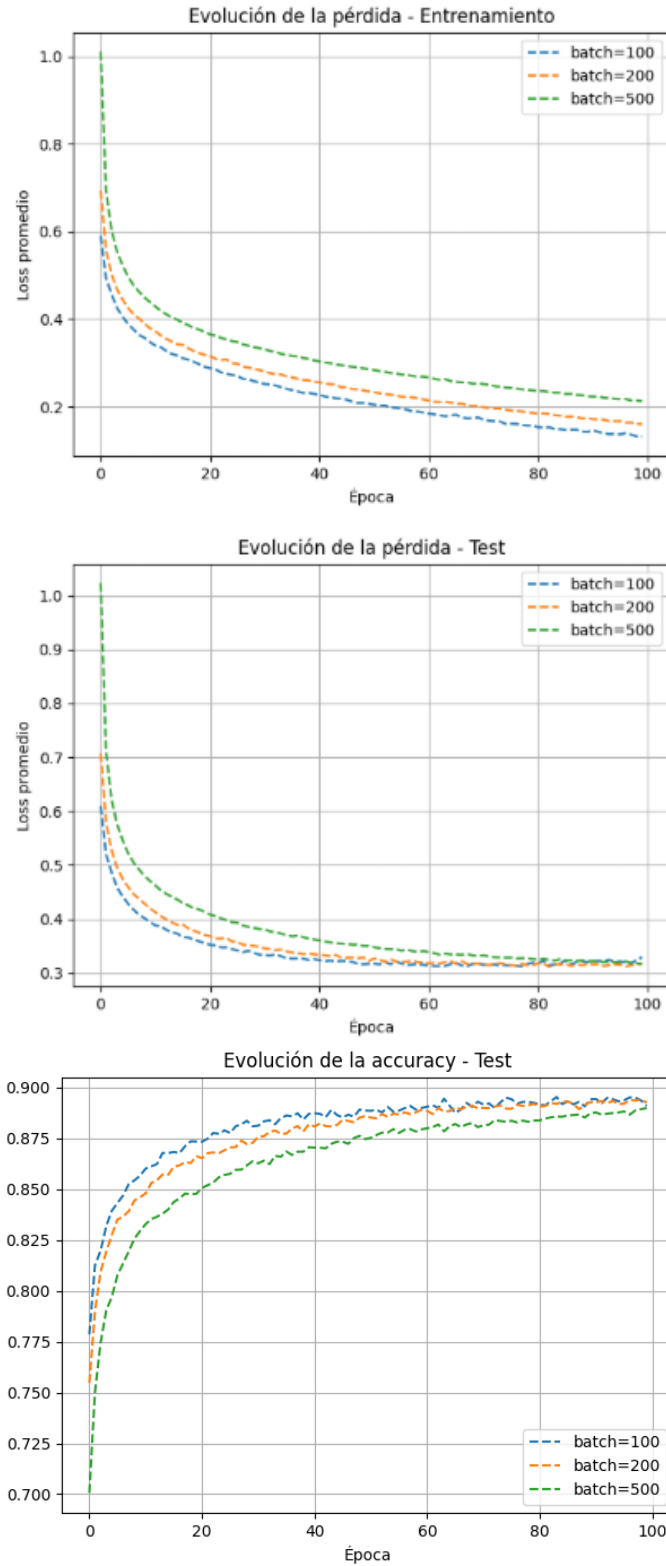


Figura 2. Diferencias entre las épocas y el tamaño del lote. En el gráfico se observa cómo varía el comportamiento del modelo frente a distintos tamaños de lote. Recordemos que el *batch size* corresponde a la cantidad de datos procesados simultáneamente durante el entrenamiento. Para valores de lote más grandes, el aprendizaje tiende a ser más lento debido a que se realizan menos actualizaciones de los pesos por época. Además, podemos observar que a medida que se agranda el tamaño del lote, el error inicial es cada vez mas grande.

C. Experimento 3

En este experimento, probaremos distintos tipos de dropouts. Que son la cantidad de neuronas que se apagan durante el entrenamiento. Este proceso se da durante el entrenamiento y es uno de los tantos métodos utilizados para la regularización. Para este experimento, utilizamos el mejor valor encontrado en el anterior.

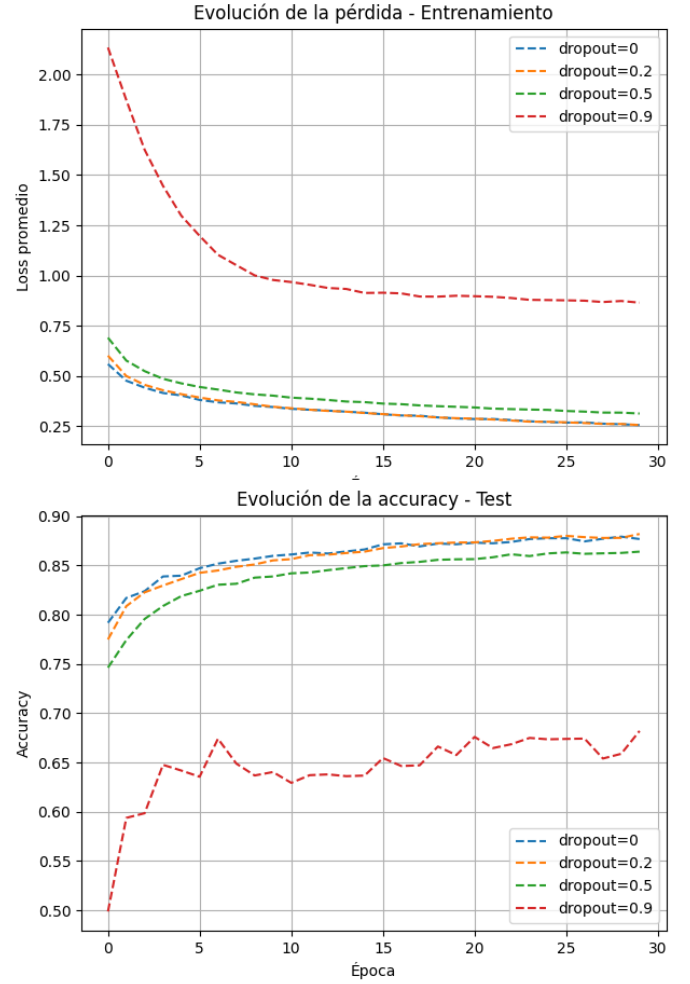


Figura 3. Impacto del **dropout** en el entrenamiento y prueba. Para cantidades altas, el modelo se ve afectado de manera negativa por el dropout, debido a la cantidad de neuronas inhibidas empeorando su entrenamiento. Para valores mas pequeños, el entrenamiento no se ve afectado, y la generalización se ve correcta. Recordemos que el dropout es una forma de evitar el subajuste.

D. Experimento 4

En este experimento, cambiaremos la cantidad de neuronas que tiene nuestra estructura. Siempre acompañadas de las mejoras obtenidas en los experimentos anteriores.

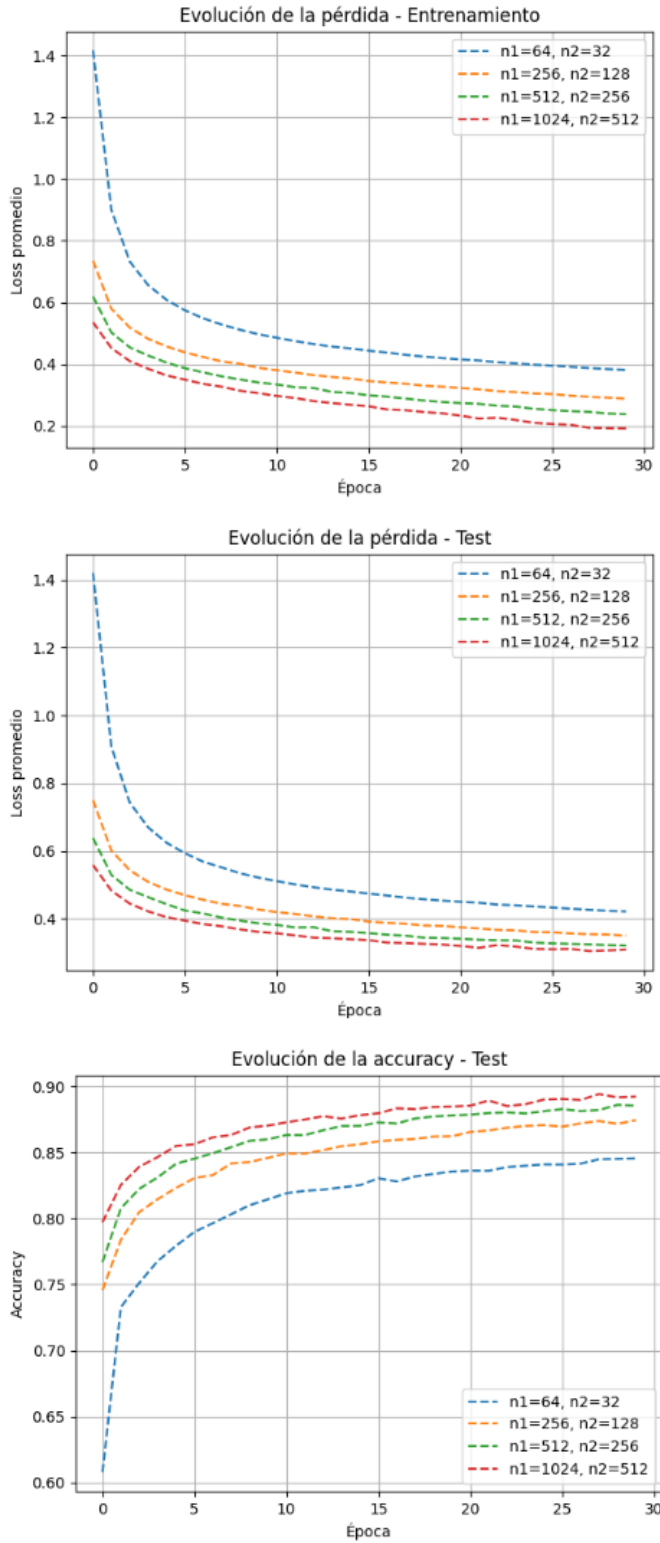


Figura 4. Comparación del desempeño del modelo para distintos tamaños de capas ocultas. A medida que aumenta el número de neuronas (n_1, n_2), el modelo logra una pérdida menor y una mayor precisión, ya que dispone de más parámetros para representar las relaciones en los datos. Sin embargo, un tamaño excesivo de red incrementa el costo computacional y puede llevar al sobreajuste. En nuestro caso las curvas de entrenamiento de nos brindan la pista que que el modelo no esta sobreajustando.

E. Modelo Final

Luego de haber hecho un ajuste de hiperparametros con nuestros experimentos, estamos en condiciones de entrenar un modelo capaz de llevar a cabo una mejor generalización.

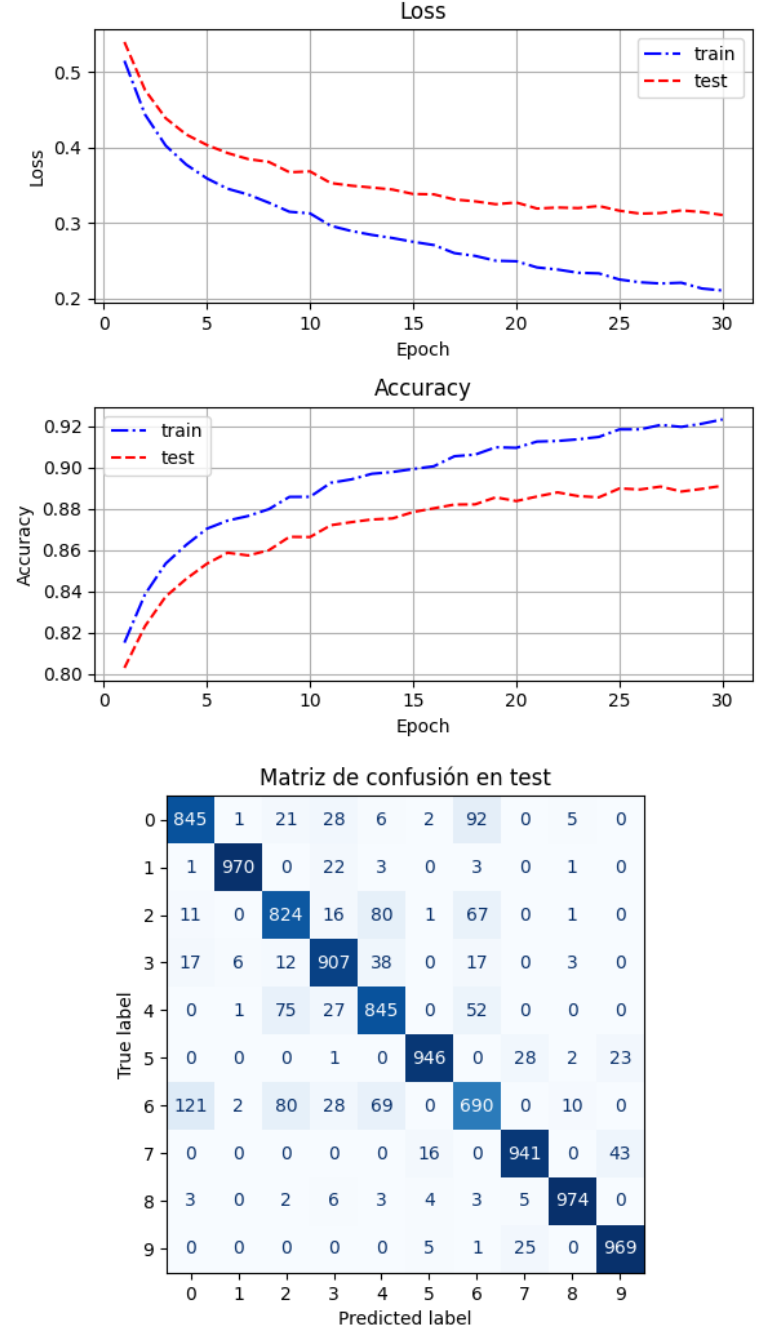


Figura 5. Resultados del la red neuronal entrenada con los mejores hiperparametros encontrados durante la realización de nuestros experimentos.

En la Figura 1, Podemos ver que cuando la tasa de aprendizaje es muy alta, se crea un problema de divergencia

temporal de los gradientes, los pesos w cambian demasiado bruscamente de un paso al siguiente. En vez de acercarse suavemente al mínimo de la función de pérdida $L(w)$, saltan por encima del valle, aumentándolo momentáneamente. En el caso de los learning rate muy bajos, se toma mucho tiempo en llegar a converger al mínimo. Por lo tanto hacen falta muchas más épocas para llegar a este, la consecuencia de esto es que nuestro modelo va a sufrir de subajuste. Esto lo podemos ver en nuestras curvas de testeo, donde vemos los resultados de probar nuestro modelo con datos nunca vistos antes. Para las curvas de las tasas de aprendizaje muy altas y muy bajas quedan en evidencia el subajuste que mencionamos anteriormente.

En la Figura 2, Se observa que, a medida que el tamaño del lote aumenta, la convergencia se vuelve más lenta. Esto se debe a que los gradientes calculados con lotes grandes son más estables pero se actualizan con menor frecuencia, reduciendo la variabilidad estocástica que puede favorecer el aprendizaje haciendo que la generalización del modelo sea peor. Además de esto, consumen mucha más memoria que un lote pequeño. Por el contrario, los lotes pequeños permiten actualizaciones más ruidosas pero más rápidas, lo que acelera la disminución de la pérdida y el aumento de la accuracy en las primeras épocas. En este caso, el tamaño de lote 100 ofrece un buen equilibrio entre velocidad de aprendizaje y rendimiento final.

En la Figura 3 se observa cómo el porcentaje de dropout afecta el proceso de aprendizaje del modelo. Este método desactiva de manera aleatoria una fracción de las neuronas durante el entrenamiento, lo que ayuda a reducir el sobreajuste al evitar que el modelo dependa excesivamente de conexiones específicas. Sin embargo, valores de dropout demasiado altos reducen drásticamente la capacidad de aprendizaje del modelo, ya que se eliminan demasiadas neuronas activas en cada iteración. En los resultados mostrados, se aprecia que configuraciones con dropout elevados presentan una pérdida de entrenamiento más lenta y una generalización significativamente peor que con otras configuraciones.

En la Figura 4, aumentar simultáneamente el tamaño de la primera capa (n_1) y la segunda capa (n_2) resulta en un mejor rendimiento general, mejora el ajuste en el entrenamiento ya que la pérdida es menor y mejora la generalización donde la precisión al probar nuestro modelo es mayor. La configuración $n_1 = 1024, n_2 = 512$ ofrece el mejor equilibrio y rendimiento máximo, sugiriendo que la capacidad del modelo era un factor limitante y su aumento mejoró la capacidad de la red para aprender las características relevantes y aplicarlas a datos nuevos pudiendo mostrar un sobreajustar no muy significativo.

Finalmente, en la Figura 5, tenemos el entrenamiento y prueba de nuestro modelo final encontrado. Los hiperparámetros fueron:

- Tasa de aprendizaje: 0.0001
- Cantidad de capas ocultas: 2
- Cantidad de neuronas en la primera capa oculta: 512
- Cantidad de neuronas en la segunda capa oculta: 256
- Dropout: 0.5
- Optimizador: Adam
- Tamaño de los lotes: 100
- Epocas: 30

En base a estos hiperparámetros, encontrados mediante un ajuste hecho en base a los experimentos descritos en este informe. Obtuvimos un modelo que tiene un **accuracy promedio del 89,11 %**, el cual carece de **overfitting** significativo, mostrando una generalización correcta. Se muestra también, la matriz de confusión del modelo, donde podemos ver que para determinadas etiquetas, las predicciones casi fueron perfectas.

V. DISCUSIÓN

El modelo MLP implementado logró un desempeño satisfactorio en la clasificación de imágenes de ropa, alcanzando un **accuracy promedio del 89,11 %**. La reducción constante de la función de pérdida y la proximidad entre las curvas de entrenamiento y prueba indican una correcta convergencia y ausencia de sobreajuste significativo.

A pesar de no incorporar capas convolucionales, la red multicapa fue capaz de capturar patrones relevantes de los datos de entrada. Las principales confusiones se produjeron entre clases con alta similitud estructural, por lo que se espera que un modelo con convoluciones (CNN) o técnicas de *data augmentation* pueda superar los resultados obtenidos. Con respecto al rendimiento con gran cantidad de neurona por capas, este lleva al overfitting. Es necesario aplicarle un **dropout** más grande o, como en este caso, disminuir la cantidad de neuronas. Otra técnica posible es el **Early Stopping**, que se basa en cortar el entrenamiento en el momento adecuado, antes que aparezcan síntomas de overfitting.

En síntesis, el trabajo permitió implementar y analizar un modelo de clasificación basado en descenso por gradiente en PyTorch, comprendiendo de manera práctica la relación entre optimización, arquitectura y desempeño general del modelo.

-
- [1] W. Samek, T. Wiegand, and K.-R. Müller, “Robust Explainability: A tutorial on gradient-based attribution methods for deep neural networks,” *IEEE Signal Processing Magazine*, vol. 39, no. 6, pp. 93–114, 2022. doi: 10.1109/MSP.2022.3201065.
 - [2] McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115-133.
 - [3] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” *arXiv preprint arXiv:1708.07747*, 2017. Available: <https://arxiv.org/abs/1708.07747>
 - [4] A. F. Agarap, “Deep Learning using Rectified Linear Units (ReLU),” *arXiv preprint arXiv:1803.08375*, 2018. Available: <https://arxiv.org/abs/1803.08375>
 - [5] PyTorch Contributors, “Softmax — torch.nn.Softmax,” *PyTorch 2.9 Documentation*, Accessed: 2025-11-12. Available: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Softmax.html>
 - [6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986. <https://doi.org/10.1038/323533a0>
 - [7] A. Mao, M. Mohri, and Y. Zhong, “Cross-Entropy Loss Functions: Theoretical Analysis and Applications,” *arXiv preprint arXiv:2304.07288*, 2023. Available: <https://arxiv.org/abs/2304.07288>
 - [8] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn and TensorFlow*. 1st ed. O’Reilly Media, 2017. ISBN 9781491962299.